

DEEP LEARNING

Deep Reinforcement Learning: Part I

Subhankar Roy

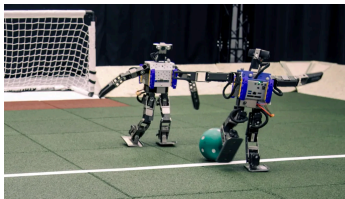
Department of Management, Information and Production Engineering (DIGIP)
University of Bergamo

Motivation

- Many real-world problems are not about predicting labels – they involve making decisions.
 - ▶ An "agent" (say robot) must act (take an action), not just predict labels
 - ▶ Actions affect the future state of the world
 - ▶ Feedback is often delayed or sparse
 - ▶ There is no "correct label" for each action (eg, Chess)



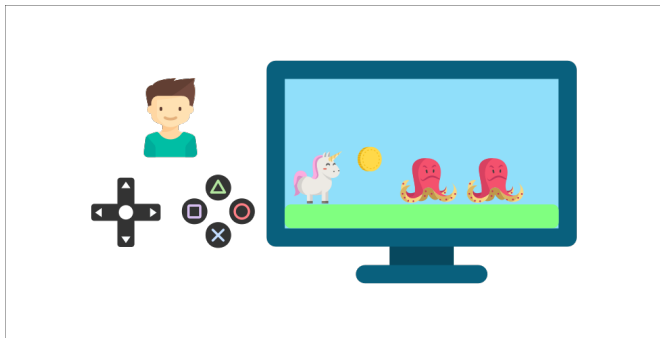
A rescue robot navigating through a site



A pair of robots playing football

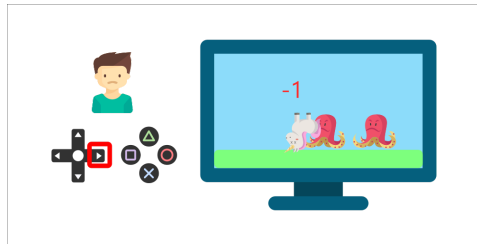
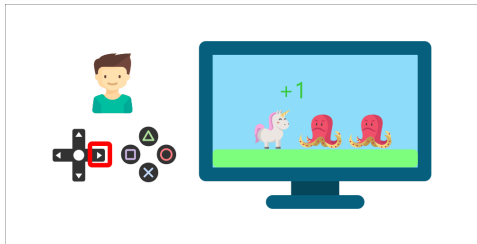
- We need a framework for learning how to act when outcomes depend on a sequence of decisions → **Reinforcement Learning**

What is Reinforcement Learning?



- Imagine that you are playing a particular video game for the *first* time ever and nobody told you the rules.
- How do you start to learn playing this game?

What is Reinforcement Learning?



- You press button(s) and check the outcomes.
 - ▶ You press "move right" once and you get a coin (+1).
 - ▶ You press "move right" another time; and this time the unicorn touches the enemy and it dies, and you lose a coin (-1)!
- By interacting with this environment, you learn to get coins in this environment while avoiding enemies. Therefore, you learn to play this game by **trial and error**.

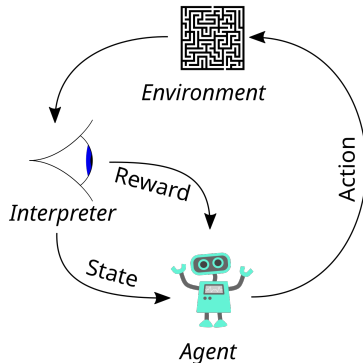
What is Reinforcement Learning?

- **Reinforcement Learning** (RL) is a framework for solving control tasks (or called decision problems) by building agents that learn from the environment by interacting with it through trial and error and receiving rewards (positive or negative) as unique feedback.
- In other words, RL is just a computational approach of learning from actions.
- Real-world applications of RL¹:
 - ▶ Training agents that play games like Chess and Go.
 - ▶ In autonomous driving it helps decide when to accelerate, brake or change lanes.
 - ▶ Adjusting prices in real time (eg, ride-sharing fares, airline tickets).
 - ▶ Robots learn the best paths to pick and deliver items efficiently.
 - ▶ Fine-tune LLMs like ChatGPT so its answers match human preferences and expectations.

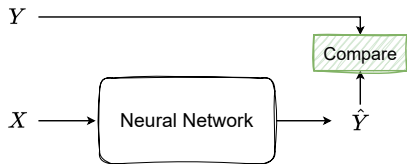
¹Paper on RL applications

Terminologies in Reinforcement Learning

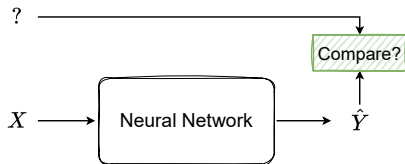
- **Agent**: a learner or a decision-maker. For example, a robot or a program trying to figure out what to do.
- **Environment**: everything outside the agent that it interacts with. For example, a video game or maze.
- **Action**: the choices an agent can make. For example, move left, jump or do nothing.
- **State**: a snapshot of the situation the agent is currently in. For example, where the agent is or how much battery left.
- **Reward**: feedback that tells the agent how well it did. Positive reward = good job, negative reward = bad move.



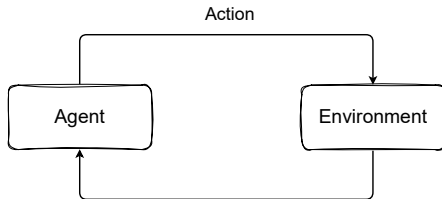
Comparing Learning Paradigms



Supervised Learning



Unsupervised Learning



Reinforcement Learning

Today

- Learn how to teach machines to act, say play video games.
- Introduce the basics of reinforcement learning.
- Traditional ways of solving reinforcement learning, such as Q-learning.
- Modern ways of solving reinforcement learning, which include
 - ▶ Deep Q-learning
 - ▶ Policy-gradient methods

Table of Contents

1. Basics of Reinforcement Learning

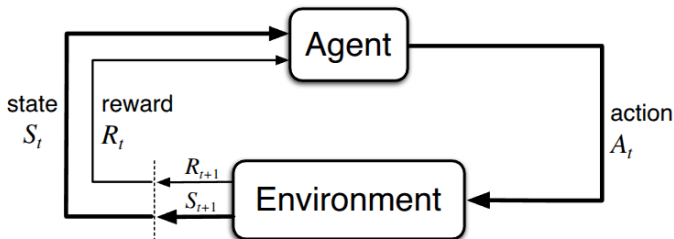
2. Q-Learning

3. Deep Q-Learning

4. Policy Gradient

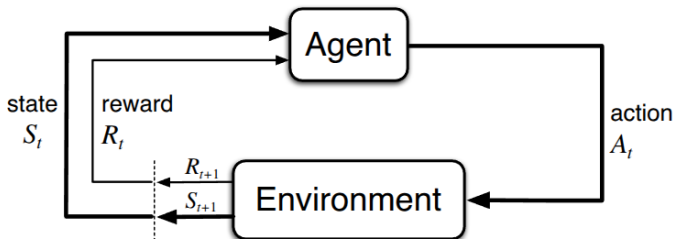
Basics of Reinforcement Learning

Reinforcement Learning Framework



- The learner and decision maker is called the *agent*. The agent interacts with the *environment*.
- The agent and the environment interact continually, the agent selecting actions and the environment responding to these actions and presenting new situations to the agent.
- The environment also gives rise to rewards (numerical values) that the agent seeks to *maximize* over time through its choice of actions.

Reinforcement Learning Framework

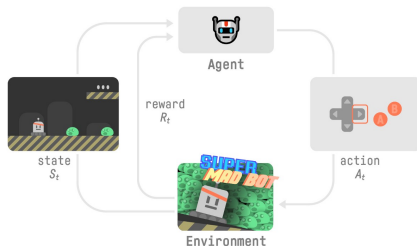


- Interactions happen over discrete time steps, $t = 0, 1, 2, 3, \dots$. At each time step t :
 - ▶ The agent receives some representation of the environment's **state**, S_t
 - ▶ Depending on S_t , the agent selects an action A_t .
- One time step later, ie, at $t + 1$:
 - ▶ The agent receives a numerical reward R_{t+1} and
 - ▶ Finds itself in a new state S_{t+1}
- It gives rise to a sequence or **trajectory** that begins like:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$$

Reinforcement Learning Framework: An Example

- An agent receives state S_t from the environment, ie, the frame of the game at time step t .
- Based on S_t , the agent takes action A_t , ie, move to the right.
- The environment goes to a new state S_{t+1} , ie, a new frame at time step $t + 1$.
- The environment also gives some reward R_{t+1} to the agent, ie, we are not dead (a positive reward $+1$).



Reward Hypothesis in RL

- The agent's goal is to *maximize* its cumulative rewards, called the **expected return**, in the **long run**.
- RL's main idea is based on the **reward hypothesis**, which is that all goals can be described as the maximization of the expected return. To have the best behaviour, we aim to learn to take actions that maximize the expected cumulative reward.
- The cumulative reward of a trajectory at a given time step t is given as:

$$\begin{aligned} G_t &= R_{t+1} + R_{t+2} + R_{t+3} + \dots \\ &= \sum_{k=0}^{\infty} R_{t+k+1}, \end{aligned}$$

where $R_{t+1}, R_{t+2}, \dots$ are the sequence of rewards received after time t .

Reward Hypothesis in RL

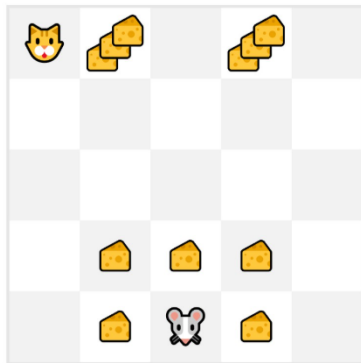
- Not all rewards are equal, which brings us to the idea of **discounted rewards**.
- In this approach, the agent tries to select actions so that the sum of the discounted rewards it receives over the future is maximized.
- It chooses A_t to maximize the expected discounted return:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

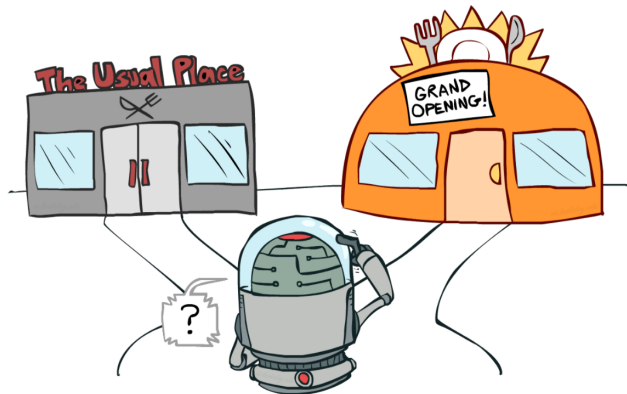
where γ is a parameter $0 \leq \gamma \leq 1$, called the discount rate.

Reward Hypothesis in RL: An Example

- Consider that the mouse's goal is to eat the maximum amount of cheese before being eaten by the cat.
- The reward near the cat, even if its bigger (more cheese), will be more discounted since we are not really sure we will be able to eat it.
- Therefore, the γ will be set close to zero, as $\gamma^1, \gamma^2, \gamma^3, \dots$ drops to near zero real fast.
- When γ is close to 1, we also care about long-term rewards.



The Exploration/Exploitation Trade-off



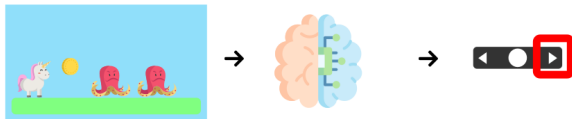
- Given the choice of picking a restaurant where you know the food is good everyday *versus* a newly opened restaurant where you do not know how the food tastes, which one will you pick?

The Exploration/Exploitation Trade-off

- This "dilemma" is known as the [exploration/exploitation trade-off](#), and is an important aspect in RL.
- In the example of the restaurant, there is a trade-off between:
 - ▶ [Exploitation](#): You go to the same one that you know the food is good everyday but you risk to miss a potentially better restaurant.
 - ▶ [Exploration](#): Try restaurants you never went to before, with the risk of having a bad experience, but also a probable opportunity of a fantastic experience.
- Even in video games, you face the same dilemma of earning coins from well known locations and going to unknown locations with the risk of your character getting killed, but finding a gold mine of coins!
- Generally speaking, the RL agent must balance exploration and exploitation to maximize the expected cumulative reward.

Policies

- **Policy**: The policy π is the brain of the agent. Its a function that tells us what action to take given the state we are in. Thus, it defines the agent's behaviour at a given time.
- In a video game, consider the state is "enemy nearby, low health", then the policy is "run away".
- Formally, a policy is a mapping from **states** to **probabilities** of selecting each possible **action**. If an agent is following policy π at time t , then $\pi(a | s)$ is the probability that $A_t = a$ if $S_t = s$. π is learned through training.



State $\rightarrow \pi(\text{State}) \rightarrow \text{Action}$

Value Functions

- All RL algorithms involve estimating **value functions** – functions of states (or of state-action pairs) that estimate *how good* it is for the agent to be in a given state.
- In other words, it tells the agent how good a situation or action is, in terms of future rewards.
- Two kinds of value functions:

1. **State-value function**. How much total reward can we expect starting from state s if we follow policy π :

$$v_{\pi}(s) = \mathbb{E}[G_t \mid S_t = s]$$

For eg, in a video game, "a position near the goal" $\rightarrow$ high value, "a trap" $\rightarrow$ low value.

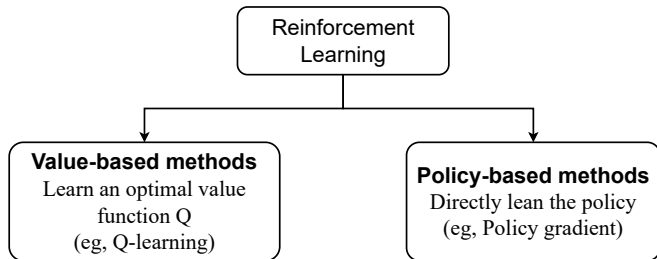
2. **Action-value function**. How good it is to take an action a in state s , and then follow policy π :

$$q_{\pi}(s, a) = \mathbb{E}[G_t \mid S_t = s, A_t = a]$$

For eg, turning left might be better than turning right $\rightarrow$ higher Q-value.

Finding The Optimal Policy

- Solving an RL task means, finding a policy that achieves a lot of reward over the long run.
- There are two approaches to train the agent to find the optimal policy π^* :
 1. **Policy-based methods**. Directly, by teaching the agent to learn which action to take, given which state the agent is in.
 2. **Value-based methods**. Indirectly, teach the agent to learn which state is more valuable and then take the action that leads to the more valuable states.



Policy-based Methods

- In Policy-based methods, we learn a policy function directly. This function will define a mapping from each state to the best corresponding action or a probability distribution over the set of possible actions at that state.
- There are two types of policies:
 1. **Deterministic**. A policy at a given state will always return the same action:

$$a = \pi(s)$$

For eg in a 2D game, "you see a coin to the right", the action is "move right".

2. **Stochastic**. Outputs a probability distribution over the actions:

$$\pi(a \mid s) = P[A \mid s]$$

For eg, "you see a coin to the right but also an enemy approaching", the distribution of actions "left: 0.1, right: 0.3, jump: 0.6".

Value-based Methods

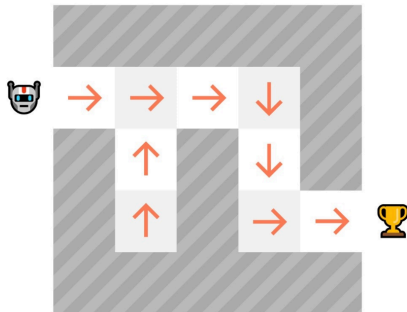
- In value-based methods, instead of learning a policy function, we learn a value function that maps a state to the expected value of being at that state.
- For eg, the agent learns: "going right gives long-term reward ≈ 10 points" and "going left gives ≈ 3 ", therefore, the action is the one with the highest coins or the highest expected (discounted) return.
- Formally, we first learn a value function ($v_\pi(s)$ or $q_\pi(s, a)$), and at decision time we take an action that gives the highest reward:

$$\pi^*(s) = \arg \max_a q^*(s, a)$$

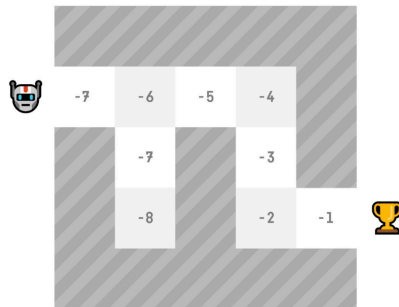
Consequently, finding an optimal value function leads to having an optimal policy.

- In contrary, in policy-based methods, the agent learn to take an action a in state s .

Comparing Policy-based with Value-based Methods

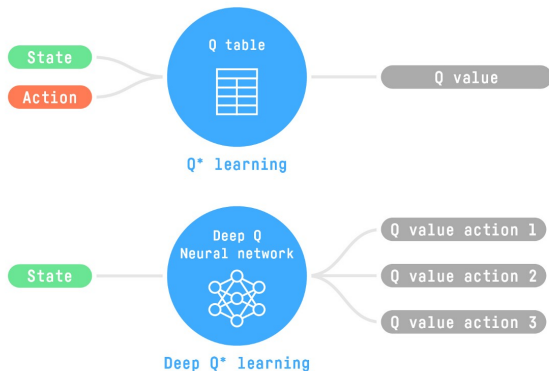


Policy-based methods



Value-based methods

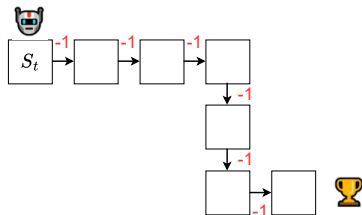
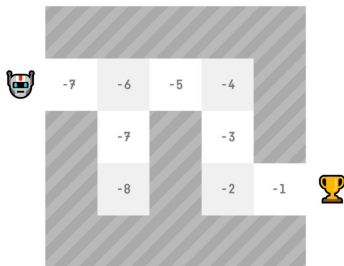
Deep Reinforcement Learning



- The word "deep" comes from the fact that we use deep neural networks to approximate the Q value, instead of traditional algorithm to create a Q table that helps us find what action to take for each state.

Q-Learning

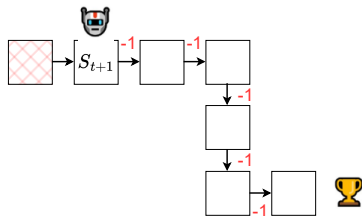
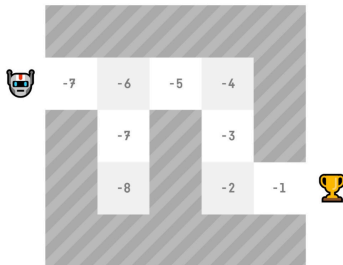
Bellman Equation



- We calculate the value of a state $S_t = s$ as the sum of the expected rewards starting at that state and then follow the policy forever after, which is:

$$v(S_t) = (-1) + (-1) + (-1) + (-1) + (-1) + (-1) = -6$$

Bellman Equation



- We calculate the sum of the expected rewards at state S_{t+1} as:

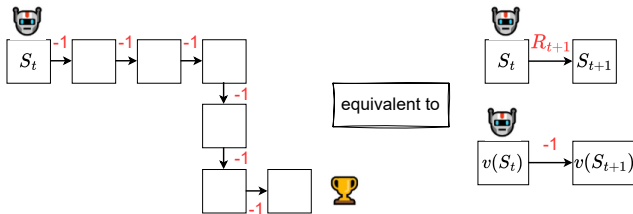
$$v(S_{t+1}) = (-1) + (-1) + (-1) + (-1) + (-1) = -5$$

- Notice that we are repeating the computation, which is not ideal!

Bellman Equation

- Instead, we will use **Bellman Equation** that will simplify our state value or state-action value calculation, given as:

$$\underbrace{v(s)}_{\text{value at state } s} = \mathbb{E}_{\pi} \left[\underbrace{R_{t+1}}_{\text{immediate reward}} + \underbrace{\gamma v(S_{t+1})}_{\text{discounted reward of next state}} \mid S_t = s \right]$$



$$v(S_t) = R_{t+1} + \gamma * v(S_{t+1}) = (-1) + 1 * (-5) = -6$$

assuming the discount rate $\gamma = 1$.

Learning Value Functions or Policies

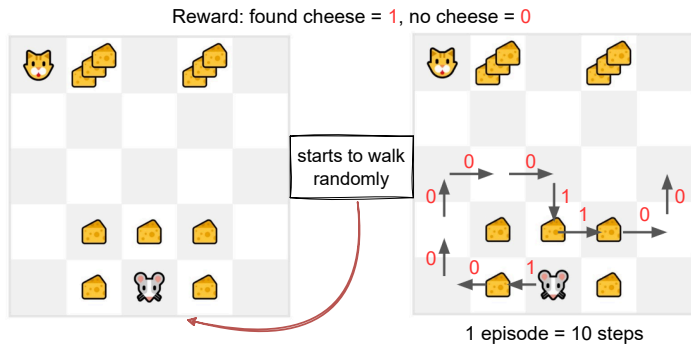
- Learning in RL means given the experience and the received reward, the agent will update its value function or policy.
- How can we *actually* learn these? We can learn using the following two strategies:
 1. **Monte Carlo**. In this approach, we wait until the end of an episode, calculate the return G_t , and use it as a target for updating $v(S_t)$.

$$\underbrace{v(S_t)}_{\text{new value at state t}} \leftarrow \underbrace{v(S_t)}_{\text{former estimation}} + \underbrace{\alpha}_{\text{lr}} \left[\underbrace{G_t}_{\text{return at t}} - \underbrace{v(S_t)}_{\text{former estimation}} \right]$$

2. **Temporal difference learning** (TD). This approach uses only a step, ie, $(S_t, A_t, R_{t+1}, S_{t+1})$ to learn. The idea is to update the $v(S_t)$ at each step.

$$v(S_t) \leftarrow v(s_t) + \alpha \underbrace{[R_{t+1} + \gamma v(S_{t+1}) - v(S_t)]}_{\text{Bellman equation}}$$

Monte Carlo



- We want to calculate $v(0)$. We will use the formula: $v(S_t) \leftarrow v(s_t) + \alpha[G_t - v(s_t)]$.
- Calculating $G_t = 1 + 0 + 0 + 0 + 0 + 0 + 1 + 1 + 0 + 0 = 3$.
- New $v(0) = v(0) + \text{lr} * [G_t - v(0)] = 0 + 0.1 * 3 = 0.3$.

Temporal Difference



- We initialize our value function so that it returns 0 value for each state. We will update the value function after any individual step $\rightarrow$ one-step TD.
- We want to calculate $v(0)$ using $v(S_t) \leftarrow v(s_t) + \alpha[R_{t+1} + \gamma v(S_{t+1}) - v(S_t)]$.
- New $v(0) = v(0) + \text{lr} * [R_1 + \gamma * v(1) - v(0)] = 0 + 0.1 * [1 + 1 * 0 - 0] = 0.1$.

Q-Learning



- Q-Learning is a value-based method that uses a TD approach to train its action-value function.
- We are training a Q-function, an action-value function that determines the value of being at a particular state and taking a specific action at that state.
- Internally, the Q-function is a table, where each cell correspond to a state-action pair value.

Q-Learning

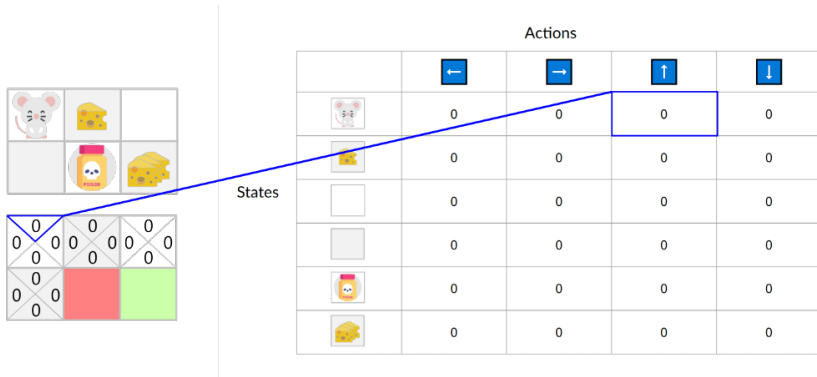
- Trains a Q-function (an action-value function), which is internally a table that contains all the state-action pair values.
- Given a state and action, our Q-function will search its Q-table for the corresponding value.
- When the training is done, we have an optimal Q-function, which means we have optimal Q-table.
- If we have an optimal Q-function, we have an optimal policy since we know the best action to take at each state.

Q-Learning: An Example



- The mouse is in a maze where there is a small cheese, poison and big pile of cheese. The goal: eat the big pile of cheese and avoid poison.
- The episode ends if the mouse eats the poison, eat the big pile of cheese or if it spends more than 5 steps.
- $\text{lr} = 0.1$, $\gamma = 0.99$.

Q-Learning: An Example (Step 1)

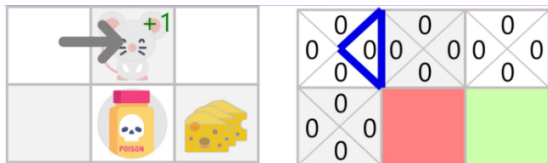


- The Q-table has a value of each state-action pair. It is initialized to zeros, which is the first step of the learning algorithm.
- During training, the agent (our mouse) explores the environment and we update the Q-table.

epsilon-greedy Policy

- Before we go into the Q-learning algorithm to estimate the Q-table, we need a policy, which is the one responsible for taking actions.
- We select **epsilon-greedy** (ϵ -greedy) strategy as a policy to handle the exploration/exploitation trade-off.
- The algorithm works as follows:
 - ▶ With probability $1 - \epsilon$ we do exploitation (or greedy action), ie, the agent selects the action with the highest state-action pair value.
 - ▶ With probability ϵ we do exploration, ie, trying random action.
- During training, we start with a high value of $\epsilon = 1.0$ to encourage exploration. Then it is decreased exponentially over time as the Q-table gets better and better in its estimations.

Q-Learning: An Example (Step 2)



- Because epsilon is big ($\epsilon = 1.0$), the agent will take a random action: move right and go to a new state.
- It will get a reward of $+1$ since it ate a small cheese.
- We use the equation below to calculate the new Q-value estimate:

$$q(S_t, A_t) \leftarrow q(S_t, A_t) + \alpha [R_{t+1} + \underbrace{\gamma \max_a q(S_{t+1}, a)}_{\text{discounted Q-value of next state}} - q(S_t, A_t)]$$

Q-Learning: An Example (Step 2)

- Following the formula we compute the Q-value of the new state-action pair:

$$q(\text{State 1, Right}) = 0 + 0.1 * [1 + 0.99 * 0 - 0] = 0.1$$

- We populate this value in the corresponding cell of the Q-table.

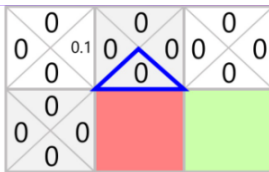


	←	→	↑	↓
🐭	0	0.1	0	0
🧀	0	0	0	0
□	0	0	0	0
■	0	0	0	0
💀	0	0	0	0
🧀	0	0	0	0

Q-Learning: An Example (Step 3)

- In the next step although the epsilon is decayed, it is still big ($\epsilon = 0.99$). Assume that the action taken: down.
- This is not a good action and the agent dies.
- We compute the Q-value of the new state-action pair and populate the table:

$$q(\text{State 2, Down}) = 0 + 0.1 * [-10 + 0.99 * 0 - 0] = -1$$



	←	→	↑	↓
☠	0	0.1	0	0
🧀	0	0	0	-1
□	0	0	0	0
■	0	0	0	0
🍷	0	0	0	0
🧀	0	0	0	0

Q-Learning: An Example

				
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

Training →

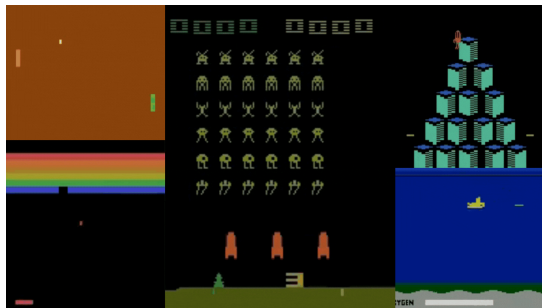
				
	0	10.8	0	0
	0	9.9	0	-10
	0	0	0	10
	0	-10	0	0
	0	0	0	0
	0	0	0	0

- Through training, we go from an arbitrary Q-table to an optimal Q-table.
- Once the Q-function (or the Q-table) is learned, do inference in the following manner:
 - ▶ The environment gives the current state s .
 - ▶ The agent looks up the row for s in the Q-table.
 - ▶ It chooses the action with the highest Q-value:

$$a^* = \arg \max_a q(s, a)$$

Deep Q-Learning

Deep Q-Learning



- Q-Learning works when the state space was discrete and small. In more complex games, like Atari, there can be 10^9 to 10^{11} states, which are too many for Q-Learning to work.
- Deep Q-Learning uses a Neural Network that takes a state and approximates Q-values for each action based on that state. It makes Q-learning tractable.

Deep Q-Learning



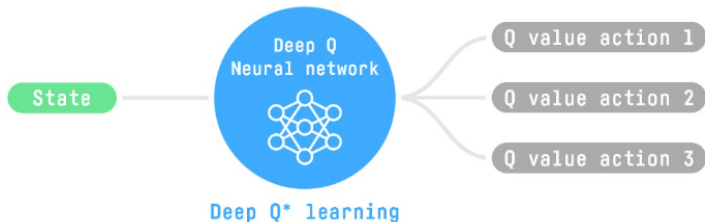
160



210

- In simple games (mouse in the maze) the total number of possible states is very small, which is equal to 9, and thus easy to model with a Q-table.
- In more complex games, such as Space Invaders, the total number of possible states is equal to $256^{160 \times 210 \times 3} = 256^{100800}$, which is more than the total number of atoms in the universe ($\approx 10^{80}$). Creating Q-table is not efficient.
- It is because each pixel can take up values between 0 – 255 and there are three channels in a RGB image.

Deep Q-Learning



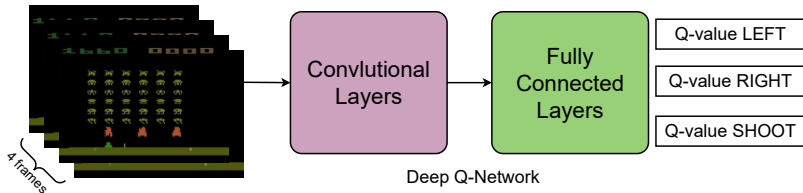
- To handle more complex games we will use Deep Q-Learning, where the key idea is to approximate Q-values using a parameterized Q-function.
- The parameterized Q-function is nothing but a (deep) neural network, which will approximate, given a state, the different Q-values for each possible action at that state.

Space Invaders

- **Goal**: destroy all the alien invaders (who can move) before they reach the bottom of the screen or destroy your defenses (with lasers).
- **Player controls** (actions): move left, move right, shoot, and do nothing.
- **Scoring** (reward): positive points for hitting an alien, negative reward for getting hit or if the alien reaches the bottom. The agent has 3 lives at the start.
- When you destroy all the aliens, a new wave starts and they move faster. The game continues until you lose all lives.



Deep Q-Network



- **Input:** a stack of four frames, which act as the state.
- **DQN:** a neural network made of convolutional layers and fully connected layers.
- **Output:** a vector of Q-values for each possible action at that state.
- The DQN estimates the Q-values given a state, which is then used by the epsilon-greedy policy to take an action.
- Four frames are stacked because one frame is not enough to have a sense of motion.

Deep Q Algorithm

- Deep Q-Learning trains the network to approximate the Q-function:

$$q(s, a; \theta) \approx \text{expected future reward}$$

- In order to train the network, we need to things:
 - ▶ A target value (TD target), which is what the Q-value *should* be.
 - ▶ A loss function, which will measure how wrong the current prediction is.
- The problems are:
 - ▶ We do not have a Q-table anymore
 - ▶ We do not have any ground-truth Q-values
- Q: How can we measure the discrepancy in prediction if we do not know what the true target is?

Deep Q Algorithm

- Lets assume that at time t , the agent is in state s , takes an action a , gets reward r and lands in the next state s' .
- We want to know "How good was action a while being in state s ?"
- But we don't know the future (what happens after s'). Therefore, we will approximate the future using the intuition: "If we end up in state s' , and from there we act optimally, how much reward can we expect?"
- We will use an approximation of the target:

$$\underbrace{y}_{\text{TD target}} = \underbrace{r}_{\text{current reward}} + \underbrace{\gamma \max_{a'} q(s', a')}_{\text{discounted Q-value of next-state } s'}$$

- Q: How do we get the score of the second term of the equation above?

Deep Q Algorithm

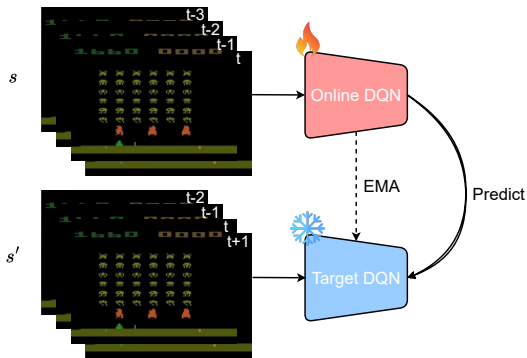
- DQN uses two neural networks with the same architecture:
 - ▶ **Online network**, the network that estimates the Q-values at the current state s , the that we actually train (with SGD) $\rightarrow q(s, a; \theta)$.
 - ▶ **Target network**, a frozen copy of the online network with parameters θ^- that estimates the Q-values of all the actions in the next state s' , and are updated slowly using EMA (recall DINO) $\rightarrow q(s', a'; \theta^-)$
- The TD target which the online network tries to approximate is:

$$y = r + \gamma \max_{a'} q(s', a'; \theta^-)$$

- If s' is the terminal state (ie, not future states are possible), we use $y = r$.
- The DQN loss is simply a MSE loss:

$$\mathcal{L}(\theta) = \mathbb{E}[(\underbrace{y}_{\text{TD target}} - \underbrace{q(s, a; \theta)}_{\text{prediction}})^2]$$

Deep Q Algorithm



- The online network (Online DQN) observes a set of frames $s = [\text{frame}_{t-3}, \text{frame}_{t-2}, \text{frame}_{t-1}, \text{frame}_t]$, while the target network (Target DQN) observes $s' = [\text{frame}_{t-2}, \text{frame}_{t-1}, \text{frame}_t, \text{frame}_{t+1}]$.

Policy Gradient

Policy Gradient

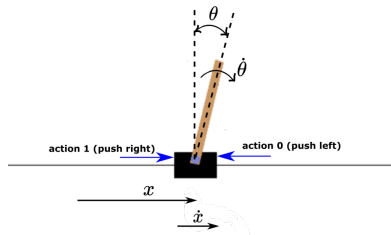
- In policy-based methods we directly learn to approximate the optimal policy π^* , without having to learn a value function, as in Q-learning.
- In particular, we parameterize the policy using a neural network π_θ , such that this learned policy will output a probability distribution over actions.

$$\pi_\theta(s) = \mathbb{P}[A \mid s; \theta]$$

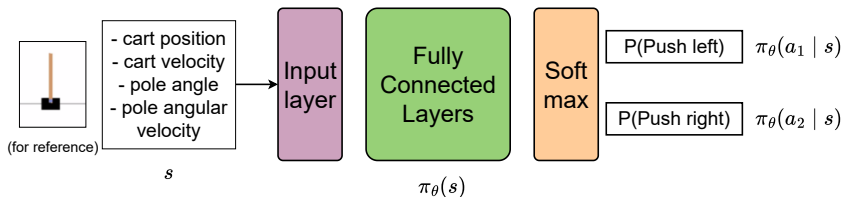
- The objective is to maximize the performance (ie, the expected return) of the parameterized policy using some optimization technique.
- **Policy gradient** methods is a subclass of the policy-based methods, where we search directly for the optimal policy by optimizing the parameters θ with *gradient ascent*.

Balancing a CartPole

- **Goal.** The agent must balance a pole upright (under gravity) on a moving cart (1D horizontal track) by applying forces to the cart left or right.
- **State.** The agent observes four continuous values: cart position, cart velocity, pole angle, pole angular velocity.
- **Action.** Push left (0) and push right (1).
- **Reward.** +1 reward for every time step the pole remains upright ($< 15^\circ$).
- Episode ends when the pole falls beyond a certain angle or the cart moves too far from center.



Policy Gradient



- Goal is to control the probability distribution of actions by tuning the policy such that good actions (that maximize the return) are sampled more frequently in the future
→ increase the probability of good actions, decrease the probability of bad ones.
- **Input:** the states s of the cart pole.
- **Output:** a distribution over the actions $\pi_{\theta}(a | s)$.
- During training, the agent π_{θ} (a fully-connected neural network) interacts with the environment and learns to tweak the parameters θ .

Policy Gradient: Collecting Trajectories

- We first start with **collecting trajectories**. Recall, a trajectory (also called an episode) is the sequence of experiences generated by running a policy in the environment.
- For the CartPole balancing task, we:
 - ▶ Start from the initial state s_0 .
 - ▶ Use the current policy to choose an action $a_t \sim \pi_\theta(\cdot \mid s_t)$.
 - ▶ Applying the action to get the next state s_{t+1} and reward r_t .
 - ▶ We store the item (s_t, a_t, r_t) , which is a tuple in the trajectory.
 - ▶ Repeat this until the episode ends.
- We end up with a trajectory, which looks like:

$$(s_0, a_0, r_0), (s_1, a_1, r_1), (s_2, a_2, r_2) \cdots$$

- With this information, we also compute the return at each time-step t :

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots$$

Policy Gradient: Reinforce

- The [Reinforce algorithm](#), also called Monte-Carlo policy-gradient, is a policy-gradient algorithm that simplifies the learning of the parameters θ .
- It works as follows:
 - ▶ Use the policy π_θ to collect an episode (or trajectory) τ .
 - ▶ Use the episode to estimate the gradient $g = \nabla_\theta J(\theta)$, where $J(\theta)$ is the objective function.
 - ▶ Update the weights of the policy: $\theta \leftarrow \theta + \alpha g$.

Policy Gradient: Objective Function

- The objective function in Reinforce for a given trajectory is:

$$J(\theta) = \sum_{t=0}^T \log \pi_{\theta}(a_t | s_t) G_t,$$

where $\pi_{\theta}(a_t | s_t)$ is the log probability of taking an action a_t in state s_t , G_t is the return at time-step t .

- The term $\log \pi_{\theta}(a_t | s_t) G_t$ means:
 - ▶ If this action eventually led to high reward, increase its probability.
 - ▶ If this action led to low reward, decrease its probability.
- Instead of maximizing $J(\theta)$, we minimize the loss (by adding a negative sign in front):

$$\mathcal{L} = - \sum_{t=0}^T \log \pi_{\theta}(a_t | s_t) G_t$$

- We repeat this on several trajectories to have a well-trained policy π_{θ} .

References and Links

- For additional reading materials refer to the following links
 - ▶ [“Deep RL Course”](https://huggingface.co/learn/deep-rl-course/en)²
 - ▶ [“Reinforcement Learning: An Introduction”](http://incompleteideas.net/book/RLbook2020.pdf)³ (Optional)

²<https://huggingface.co/learn/deep-rl-course/en>

³<http://incompleteideas.net/book/RLbook2020.pdf>